

ARQUITECTURA DE APLICACIONES WEB

Servidores, proxies, Kubernetes y el patrón MVC

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Jaime Morales

Contenido

1. El servidor y el proxy	3
1.1. Tipos de proxy	3
1.2. Nginx frente a Apache	3
2. El servidor web	3
3. Kubernetes	4
3.1. Por qué triunfan los contenedores	4
4. El patrón MVC	5
4.1. Ventajas y desventajas	5
5. Qué hemos aprendido	5

1. El servidor y el proxy

Un **servidor** es, en esencia, un ordenador (o un programa) que ofrece recursos, datos o servicios a otros equipos —los clientes— a través de una red. Antes de hablar de servidores web propiamente dichos conviene detenerse en una pieza que aparece constantemente en cualquier arquitectura: el proxy.

Un **proxy** es un intermediario que se coloca entre quien hace una petición y quien la recibe. En vez de conectar directamente con el destino, el cliente habla con el proxy, y es este quien reenvía la petición y devuelve la respuesta. Esa intermediación tiene una consecuencia inmediata: el servidor de destino solo ve la IP del proxy, no la del usuario real, lo que aporta un cierto grado de anonimato.

1.1. Tipos de proxy

No todos los proxies cumplen la misma función. Merece la pena distinguir los más habituales:

Proxy web (directo) — el propio usuario lo configura para navegar a través de él, normalmente en busca de anonimato o para saltarse restricciones geográficas.

Proxy inverso — se sitúa delante de uno o varios servidores web y reparte entre ellos las peticiones entrantes, actuando de cara al exterior como si fuera el propio servidor.

Proxy NAT — reescribe las direcciones IP de origen o destino de los paquetes, enmascarando la red interna.

Proxy interdominio — permite que tecnologías web asíncronas se comuniquen entre dominios distintos sin saltarse las restricciones del navegador.

Proxy abierto — acepta peticiones de cualquier equipo que se conecte a él, tratándolas como si fueran propias.

★ Lo más importante

- ▶ Un proxy es un intermediario que reenvía peticiones y respuestas entre cliente y servidor.
- ▶ El proxy inverso reparte tráfico entre servidores; el proxy directo protege la identidad del cliente.
- ▶ Ocultar la topología real de la red (IPs internas) es una de las grandes ventajas de seguridad de un proxy.

1.2. Nginx frente a Apache

Entre los proxies y servidores más utilizados destaca **Nginx**, creado por Igor Sysoev en 2004 con un objetivo muy concreto: resolver el llamado "problema C10K", es decir, atender miles de conexiones simultáneas sin disparar el consumo de memoria. Frente a él, **Apache** sigue siendo uno de los servidores web de propósito general más extendidos.

En rendimiento puro, Nginx suele ser más rápido y consumir menos memoria bajo alta concurrencia, especialmente sirviendo contenido estático; Apache, por su parte, cuenta con más módulos oficiales y un soporte más maduro en entornos Windows.

Ninguno de los dos es "mejor" en términos absolutos: la elección depende del tipo de tráfico, del sistema operativo del servidor y de la familiaridad del equipo con cada herramienta.

2. El servidor web

Un **servidor web** es el encargado de desplegar el backend que atiende las peticiones que llegan desde el navegador, siguiendo el clásico modelo **cliente-servidor**. Esa comunicación se produce mediante el protocolo HTTP (o su variante cifrada, HTTPS), a través de dos tipos de petición principales: **GET**, que solicita información (una página, una imagen...), y **POST**, que envía datos al servidor, como ocurre al enviar un formulario.

Existen varios servidores de aplicaciones especializados según el lenguaje o la plataforma:

Apache — servidor HTTP de propósito general, multiplataforma.

Tomcat — pensado específicamente para aplicaciones Java (Servlet, JSP).

Microsoft IIS — la opción nativa para entornos Windows.

WebLogic y **WebSphere** — servidores de aplicaciones Java EE orientados a grandes entornos empresariales, de Oracle e IBM respectivamente.

★ Lo más importante

- El modelo cliente-servidor se sustenta en peticiones HTTP/HTTPS, principalmente GET y POST.
- Cada servidor de aplicaciones tiene su nicho: Apache/Nginx para propósito general, Tomcat para Java, IIS para Windows.

3. Kubernetes

Cuando una aplicación se despliega en contenedores y crece en número de instancias, gestionarlos a mano deja de ser viable. Ahí entra **Kubernetes** (abreviado **k8s**), una plataforma de código abierto, creada por Google, que automatiza el despliegue, el escalado y la gestión de aplicaciones en contenedores.

Para entender cómo funciona conviene fijar algunos términos clave:

Master — la máquina que controla el clúster y decide dónde se ejecuta cada tarea.

Nodo — cada máquina que ejecuta realmente las cargas de trabajo asignadas por el master.

Pod — la unidad mínima de despliegue: un grupo de uno o más contenedores que comparten IP y almacenamiento.

Servicio — una capa estable que redirige el tráfico al pod adecuado, aunque este cambie de ubicación en el clúster.

Kubectl — la herramienta de línea de comandos para interactuar con el clúster.

3.1. Por qué triunfan los contenedores

La propia documentación de Kubernetes resume varias ventajas del despliegue en contenedores frente a las máquinas virtuales tradicionales: despliegues más ágiles gracias a imágenes ligeras; integración y entrega continuas, ya que la imagen es inmutable y facilita los rollbacks; separación clara entre desarrollo y operaciones; y una portabilidad total entre nubes y distribuciones, desde un portátil hasta un clúster de Google Kubernetes Engine.

★ Lo más importante

- ▶ Kubernetes automatiza el despliegue, escalado y gestión de aplicaciones en contenedores.
- ▶ Master, Nodo, Pod y Servicio son los cuatro conceptos imprescindibles para empezar.
- ▶ Los contenedores aportan portabilidad, despliegues ágiles y separación entre Dev y Ops.

4. El patrón MVC

MVC (Modelo-Vista-Controlador) es un patrón de arquitectura que separa una aplicación en tres piezas con responsabilidades muy distintas, un enfoque que ya se utilizaba en las interfaces gráficas de los años 70 y que sigue plenamente vigente hoy.

El **modelo** representa los datos y la lógica de negocio: es quien sabe cómo se guarda y recupera la información. La **vista** es la capa de presentación, lo que ve y con lo que interactúa el usuario. Y el **controlador** actúa de intermediario, recibiendo las acciones del usuario sobre la vista, decidiendo qué hacer con ellas e invocando al modelo cuando es necesario.

4.1. Ventajas y desventajas

Separar estas tres responsabilidades hace que el código sea más limpio y mantenible, permite que distintos miembros del equipo trabajen en paralelo sobre cada capa, facilita reutilizar una misma vista para distintos modelos y simplifica enormemente las pruebas unitarias. La contrapartida es que exige ceñirse a unas convenciones concretas y su curva de aprendizaje no es inmediata, por lo que en proyectos muy pequeños puede añadir más complejidad de la que resuelve.

★ Lo más importante

- ▶ MVC separa datos (modelo), presentación (vista) y lógica de control (controlador).
- ▶ Aporta mantenibilidad y trabajo en paralelo, a cambio de una curva de aprendizaje inicial.
- ▶ En proyectos muy sencillos, puede no compensar la complejidad añadida.

5. Qué hemos aprendido

En este documento hemos recorrido las piezas que sostienen la arquitectura de una aplicación web moderna: el servidor y los distintos tipos de proxy que pueden protegerlo o repartir su carga, los servidores web y de aplicaciones más habituales, Kubernetes como orquestador de contenedores de referencia y el patrón MVC como forma de organizar el código de la propia aplicación.

Estas piezas no son excluyentes, sino complementarias: una arquitectura real combina un proxy inverso, un servidor web bien elegido, un orquestador como Kubernetes para escalar y un patrón como MVC para mantener el código ordenado a medida que el proyecto crece.

¡Enhorabuena! Documento completado